
Project Report

Topic: "University Management System"

Course Code: CSE 423

Course Title: Software Architecture

Semester: Spring 2026

Section: 01

Submitted by

Sirajam Munira Naba

2022-2-60-021

Submitted to

Ahmed Adnan

Lecturer , Department of Computer Science and Engineering

Date: 1st May, 2026

Library Management System: Applying SOLID Principles

Overview

SOLID is a set of five object-oriented design principles. The Library Management module was refactored from a single monolithic Library.java class into the libraryV2 package, applying the following SOLID principles:

SOLID	Applied
SRP	YES
OCP	YES
LSP	NO
ISP	NO
DIP	YES

1. Single Responsibility Principle (SRP)

Definition: A class should have only one responsibility.

Applied In:

Library.java (Before)

The original Library.java was a single class responsible for everything , displaying menus, taking user input, reading and writing files, managing the book list, and calculating fines. Any change to any feature required modifying this one class.

```
J LibraryManagement.java > ...
1  // package university;
2
3  import java.util.*;
4
5  public class LibraryManagement{
6
7      static Scanner input = new Scanner(System.in);
8      static Main mainClass = new Main(); // main class object
9      static Library libraryObj = new Library();
10
11
12     static void manage(String regNumber){
13         System.out.println("\n*****");
14         System.out.println("\tLibrary Management System\n");
15         System.out.println("*****");
16
17         System.out.print("1. Visit Library\n");
18         System.out.print("2. Previous Menu\n");
19         System.out.print("3. Exit\n");
20         System.out.println();
21         boolean flag = false;
22         while(true){
23             try{
24                 System.out.print("Enter your choice : ");
25                 int c = input.nextInt();
26
27                 switch(c){
28                     case 1:{
29                         flag = true;
30                         libraryObj.main(regNumber);
```

```

5  public class LibraryManagement{
12     static void manage(String regNumber){
28         case 1:{
29             flag = true;
30             libraryObj.main(regNumber);
31             break;
32         }
33         case 2 : {
34             flag = true;
35             mainClass.management(regNumber);
36             break;
37         }
38         case 3 : {
39             System.exit(0);
40             break;
41         }
42     }
43     if(flag){
44         break;
45     }
46     else{
47         System.out.print("\nInvalid input, Try Again...\n");
48         input.nextLine();
49     }
50 }
51 catch(Exception e){
52     System.out.print("\nInvalid input, Try Again...\n");
53     input.nextLine();
54 }
55 }
56 }

```

libraryV2 (After :SRP Applied)

After refactoring, each class has exactly one responsibility:

- Book.java - data model only; holds title, borrowedDate, returnDate with getters and setters
- FileBookRepository.java - file I/O only; reads and writes book records to disk
- StandardFinePolicy.java - fine calculation only; computes fine based on days borrowed
- LibraryManagement.java - menu navigation and orchestration only; calls repository and finePolicy through abstractions

J Book.java X

libraryV2 > J Book.java > {} libraryV2

```
1  package libraryV2;
2
3  import java.time.LocalDate;
4
5  public class Book {
6      private String title;
7      private LocalDate borrowedDate;
8      private LocalDate returnDate;
9
10     public Book(String title) {
11         this.title = title;
12     }
13
14     public Book(String title, LocalDate borrowedDate) {
15         this.title = title;
16         this.borrowedDate = borrowedDate;
17     }
18
19     public String getTitle() {
20         return title;
21     }
22
23     public LocalDate getBorrowedDate() {
24         return borrowedDate;
25     }
26
27     public void setBorrowedDate(LocalDate borrowedDate) {
28         this.borrowedDate = borrowedDate;
29     }
30
31     public LocalDate getReturnDate() {
32         return returnDate;
33     }
34
35     public void setReturnDate(LocalDate returnDate) {
36         this.returnDate = returnDate;
37     }
38 }
39
```

```
1  package libraryV2;
2
3  import libraryV2.interfaces.BookRepository;
4  import java.util.*;
5  import java.io.*;
6  import java.time.LocalDate;
7
8  public class FileBookRepository implements BookRepository {
9
10     private final String availableFile = "libraryV2/AvailableBooks.txt";
11     private final String borrowedFolder = "libraryV2/BorrowedBooks/";
12
13     @Override
14     public List<Book> getAvailableBooks() {
15         List<Book> books = new ArrayList<>();
16         try {
17             File file = new File(availableFile);
18             if (!file.exists())
19                 file.createNewFile();
20             Scanner sc = new Scanner(file);
21             while (sc.hasNextLine()) {
22                 String line = sc.nextLine();
23                 if (!line.isEmpty())
24                     books.add(new Book(line));
25             }
26             sc.close();
27         } catch (Exception e) {
28             System.out.println("Error reading available books.");
29         }
30         return books;
31     }
```

```

31     }
32
33     @Override
34     public void saveAvailableBooks(List<Book> books) {
35         try (FileWriter writer = new FileWriter(availableFile)) {
36             for (Book b : books) {
37                 writer.write(b.getTitle() + "\n");
38             }
39         } catch (Exception e) {
40             System.out.println("Error writing available books.");
41         }
42     }
43
44     @Override
45     public List<Book> getBorrowedBooks(String regNumber) {
46         List<Book> borrowed = new ArrayList<>();
47         try {
48             File file = new File(borrowedFolder + regNumber + ".txt");
49             if (!file.exists())
50                 return borrowed;
51             Scanner sc = new Scanner(file);
52             sc.nextLine(); // skip borrowed at date
53             while (sc.hasNextLine()) {
54                 String line = sc.nextLine();
55                 if (!line.isEmpty())
56                     borrowed.add(new Book(line, LocalDate.now().minusDays(40));
57             }
58             sc.close();
59         } catch (Exception e) {
60             System.out.println("Error reading borrowed books.");
61         }

```

Mode Ln 1, Col 1

```
J FileBookRepository.java X
libraryV2 > J FileBookRepository.java > {} libraryV2
8   public class FileBookRepository implements BookRepository {
45   public List<Book> getBorrowedBooks(String regNumber) {
61       }
62       return borrowed;
63   }
64
65   @Override
66   public void saveBorrowedBooks(String regNumber, List<Book> books) {
67       try {
68           File folder = new File(borrowedFolder);
69           if (!folder.exists())
70               folder.mkdirs();
71           File file = new File(borrowedFolder + regNumber + ".txt");
72           if (!file.exists())
73               file.createNewFile();
74           FileWriter writer = new FileWriter(file);
75           writer.write("Borrowed at: " + LocalDate.now() + "\n");
76           for (Book b : books) {
77               writer.write(b.getTitle() + "\n");
78           }
79           writer.close();
80       } catch (Exception e) {
81           System.out.println("Error saving borrowed books.");
82       }
83   }
84
85   @Override
86   public void deleteBorrowedBooks(String regNumber) {
87       File file = new File(borrowedFolder + regNumber + ".txt");
88       if (file.exists())
```



```

66     public void saveBorrowedBooks(String regNumber, List<Book> books) {
82     }
83
84
85     @Override
86     public void deleteBorrowedBooks(String regNumber) {
87         File file = new File(borrowedFolder + regNumber + ".txt");
88         if (file.exists())
89             file.delete();
90     }
91 }
92

```

J StandardFinePolicy.java X

libraryV2 > J StandardFinePolicy.java > {} libraryV2

```

1  package libraryV2;
2
3  import libraryV2.interfaces.FinePolicy;
4  import java.time.LocalDate;
5  import java.time.temporal.ChronoUnit;
6
7  public class StandardFinePolicy implements FinePolicy {
8
9      private final int maxDaysWithoutFine = 30;
10     private final int finePerDay = 50;
11
12     @Override
13     public int calculateFine(LocalDate borrowedDate, LocalDate returnDate) {
14         long daysLate = ChronoUnit.DAYS.between(borrowedDate, returnDate) - maxDaysWithoutFine;
15         return daysLate > 0 ? (int) (daysLate * finePerDay) : 0;
16     }
17 }
18

```

Why Apply SRP?

- Each class has a single, clear purpose — easier to understand and debug
- A change to fine calculation only touches StandardFinePolicy, not LibraryManagement

- Individual classes can be tested in isolation

2. Open/Closed Principle (OCP)

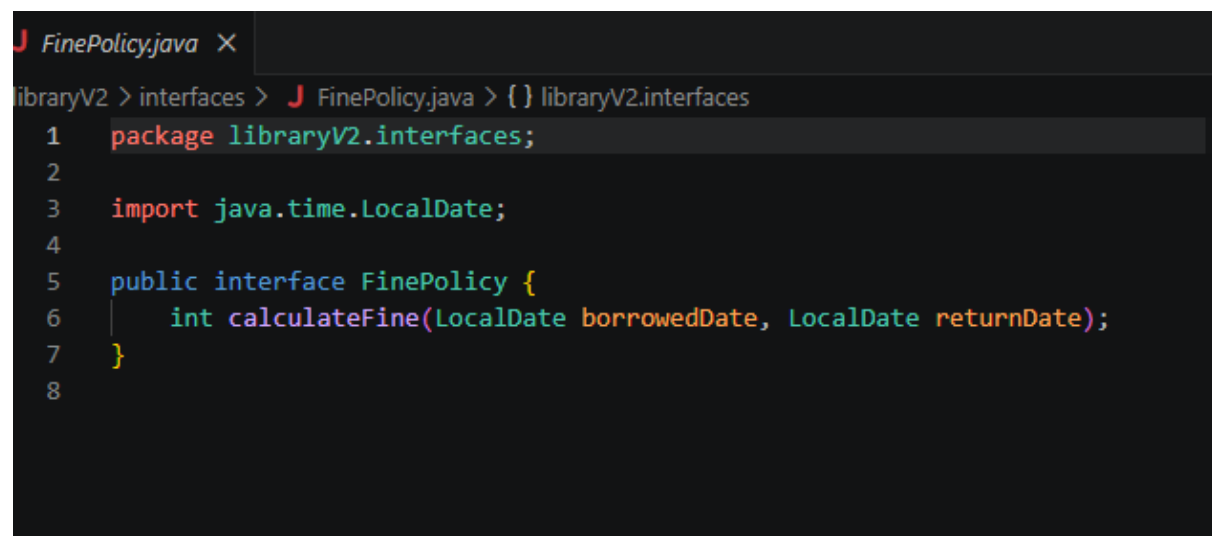
Definition: Classes should be open for extension but closed for modification.

Applied In:

FinePolicy and BookRepository Interfaces

In the original Library.java, fine logic and file operations were hardcoded directly inside the methods. Adding a different fine rate or a different storage mechanism required modifying the existing class directly.

After refactoring, both FinePolicy and BookRepository are interfaces. New implementations can be added without modifying any existing class.



```
J FinePolicy.java X
libraryV2 > interfaces > J FinePolicy.java > {} libraryV2.interfaces
1 package libraryV2.interfaces;
2
3 import java.time.LocalDate;
4
5 public interface FinePolicy {
6     int calculateFine(LocalDate borrowedDate, LocalDate returnDate);
7 }
8
```

J BookRepository.java X

libraryV2 > interfaces > J BookRepository.java > {} libraryV2.interfaces

```
1 package libraryV2.interfaces;
2
3 import java.util.List;
4 import libraryV2.Book;
5
6 public interface BookRepository {
7     List<Book> getAvailableBooks();
8
9     void saveAvailableBooks(List<Book> books);
10
11     List<Book> getBorrowedBooks(String regNumber);
12
13     void saveBorrowedBooks(String regNumber, List<Book> books);
14
15     void deleteBorrowedBooks(String regNumber);
16 }
17
```

```
J StandardFinePolicy.java X
libraryV2 > J StandardFinePolicy.java > {} libraryV2
1 package libraryV2;
2
3 import libraryV2.interfaces.FinePolicy;
4 import java.time.LocalDate;
5 import java.time.temporal.ChronoUnit;
6
7 public class StandardFinePolicy implements FinePolicy {
8
9     private final int maxDaysWithoutFine = 30;
10    private final int finePerDay = 50;
11
12    @Override
13    public int calculateFine(LocalDate borrowedDate, LocalDate returnDate) {
14        long daysLate = ChronoUnit.DAYS.between(borrowedDate, returnDate) - maxDaysWithoutFine;
15        return daysLate > 0 ? (int) (daysLate * finePerDay) : 0;
16    }
17 }
18
```

```
FileBookRepository.java X
libraryV2 > FileBookRepository.java
1 package libraryV2;
2
3 import libraryV2.interfaces.BookRepository;
4 import java.util.*;
5 import java.io.*;
6 import java.time.LocalDate;
7
8 public class FileBookRepository implements BookRepository {
9
10     private final String availableFile = "libraryV2/AvailableBooks.txt";
11     private final String borrowedFolder = "libraryV2/BorrowedBooks/";
12
13     @Override
14     public List<Book> getAvailableBooks() {
15         List<Book> books = new ArrayList<>();
16         try {
17             File file = new File(availableFile);
18             if (!file.exists())
19                 file.createNewFile();
20             Scanner sc = new Scanner(file);
21             while (sc.hasNextLine()) {
22                 String line = sc.nextLine();
23                 if (!line.isEmpty())
24                     books.add(new Book(line));
25             }
26             sc.close();
27         } catch (Exception e) {
28             System.out.println("Error reading available books.");
29         }
30         return books;
31     }
}
```

Why Apply OCP?

- New fine policies or storage backends can be added without touching existing code
- Reduces the risk of introducing bugs when extending the system

3. Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend on low-level modules. Both should depend on abstractions.

Applied In:

Library.java (Before DIP Violated)

The original `Library.java` directly instantiated file objects and called file operations (`FileWriter`, `Scanner`) scattered throughout its high-level methods. The high-level borrow and return logic was tightly coupled to low-level file system details.

```
// BEFORE — high-level method directly depends on low-level file I/O
public static void borrowBook(String regNumber) {
    File file = new File("Library/AvailableBooks.txt"); // concrete
    Scanner sc = new Scanner(file);                    // concrete
    // ...
    FileWriter fileWriter = new FileWriter(            // concrete
        "Library/BorrowedBooks/" + regNumber + ".txt", true);
    fileWriter.write("Borrowed at: " + borrowedDate + "\n");
    for (String book : borrowedbooks) {
        fileWriter.write(book + "\n");
    }
}
```

libraryV2 (After DIP Applied)

`LibraryManagement.java` now depends only on abstractions — the `BookRepository` and `FinePolicy` interfaces. The concrete classes `FileBookRepository` and `StandardFinePolicy` are assigned once at the field level. `LibraryManagement` never references them by their concrete type anywhere in its logic.

```
// AFTER — LibraryManagement depends on abstractions, not concrete classes
public class LibraryManagement {

    // Abstraction — not FileBookRepository directly in method bodies
    private static final BookRepository repository = new FileBookRepository();
    // Abstraction — not StandardFinePolicy directly in method bodies
    private static final FinePolicy finePolicy = new StandardFinePolicy();

    private static void borrowBook(String regNumber) {
        // No FileWriter, no Scanner — only the interface
        List<Book> available = repository.getAvailableBooks();
        repository.saveBorrowedBooks(regNumber, borrowed);
        repository.saveAvailableBooks(available);
    }

    private static void returnBook(String regNumber) {
        List<Book> borrowed = repository.getBorrowedBooks(regNumber);
        // No date parsing code — only the interface
        int fine = finePolicy.calculateFine(borrowedDate, returnDate);
    }
}
```

Why Apply DIP?

- `LibraryManagement` is fully decoupled from the file system — replacing `FileBookRepository` with a `DatabaseBookRepository` requires zero changes to `LibraryManagement`
- Increases flexibility and testability
- Both the high-level module (`LibraryManagement`) and low-level modules (`FileBookRepository`, `StandardFinePolicy`) now depend on the same abstractions

Refactoring code smells

LibraryManagement.java:

1. Long Method

Example method: borrowBook()

It does multiple things:

- Displaying the list of all available books
- Collecting which books the user wants to borrow
- Saving borrowed books and updating the available books file

```
public class LibraryManagement {

    private static void borrowBook(String regNumber) {
        System.out.println(x: "*****");
        System.out.println(x: "Available books are: ");

        List<Book> available = repository.getAvailableBooks();
        if (available.isEmpty()) {
            System.out.println(x: "!!!!!! No Record Found !!!!!!\n");
            return;
        }

        for (int i = 0; i < available.size(); i++) {
            System.out.println((i + 1) + ". " + available.get(i).getTitle());
        }

        System.out.println(x: "*****");

        boolean valid = false;
        while (!valid) {
            try {
                System.out.print(s: "How many books you want to borrow: ");
                int numBooks = input.nextInt();
                input.nextLine();

                if (numBooks > 0 && numBooks <= available.size()) {
                    List<Book> borrowed = new ArrayList<>();
                    for (int i = 0; i < numBooks; i++) {
                        System.out.print(s: "Which book you want to borrow, Enter its number: ");
                        int index = input.nextInt() - 1;
                        input.nextLine();
                    }
                }
            } catch (Exception e) {
                System.out.println(s: "Invalid input. Please enter a valid number.");
            }
        }
    }
}
```

```

        if (index >= 0 && index < available.size()) {
            borrowed.add(available.get(index));
        } else {
            System.out.println(x: "Invalid Book Number");
            i--;
        }
    }

    // Save borrowed books
    repository.saveBorrowedBooks(regNumber, borrowed);
    available.removeAll(borrowed);
    repository.saveAvailableBooks(available);

    System.out.println("Successfully Borrowed " + numBooks + " books");
    valid = true;
} else {
    System.out.println("Invalid Input! Only books " + available.size() + " are available.")
}
} catch (Exception e) {
    System.out.println(x: "Invalid input! Try Again");
    input.nextLine();
}
}
}

```

Refactoring:

Remedy Used: Extract Method

The method was split into three focused methods. `borrowBook()` now only coordinates the steps. The two extracted helpers each do exactly one job.

Extracted method 1: displayAvailableBooks()

```

// SMELL #6: Extracted helper – only responsible for displaying the book list
private static void displayAvailableBooks(List<Book> available) {
    for (int i = 0; i < available.size(); i++) {
        System.out.println((i + 1) + ". " + available.get(i).getTitle());
    }
    System.out.println(x: "*****");
}

```

Extracted method 2: selectBooksFromList()


```
// SMELL #6: Extracted helper – only responsible for collecting the user's picks
private static List<Book> selectBooksFromList(List<Book> available, int numBooks) {
    List<Book> selected = new ArrayList<>();
    for (int i = 0; i < numBooks; i++) {
        System.out.print(s: "Which book you want to borrow, Enter its number: ");
        int index = input.nextInt() - 1;
        input.nextLine();

        if (index >= 0 && index < available.size()) {
            selected.add(available.get(index));
        } else {
            System.out.println(x: "Invalid Book Number");
            i--;
        }
    }
    return selected;
}
```

borrowBook() after extraction — short and readable

```
public class LibraryManagement {
    private static void borrowBook(String regNumber) {
        System.out.println(x: "*****");
        System.out.println(x: "Available books are: ");

        List<Book> available = repository.getAvailableBooks();
        if (available.isEmpty()) {
            System.out.println(x: "!!!!!! No Record Found !!!!!!\n");
            return;
        }

        displayAvailableBooks(available); // SMELL #6: extracted from here

        boolean valid = false;
        while (!valid) {
            try {
                System.out.print(s: "How many books you want to borrow: ");
                int numBooks = input.nextInt();
                input.nextLine();

                if (numBooks > 0 && numBooks <= available.size()) {
                    List<Book> borrowed = selectBooksFromList(available, numBooks); // SMELL #6: extracted

                    repository.saveBorrowedBooks(regNumber, borrowed);
                    available.removeAll(borrowed);
                    repository.saveAvailableBooks(available);

                    System.out.println("Successfully Borrowed " + numBooks + " books");
                    valid = true;
                } else {
                    System.out.println("Invalid Input! Only " + available.size() + " books are available.")
                }
            } catch (Exception e) {
                System.out.println(x: "Invalid input! Try Again");
                input.nextLine();
            }
        }
    }
}
```

2. Dead Code

Location 1: Field declaration at top of class (line 9)

// private static final Main mainClass = new Main();

```
// =====  
// SMELL: DEAD CODE – this line is commented out and never  
// runs. The Main class no longer exists in the project.  
// =====  
// private static final Main mainClass = new Main();  
private static final BookRepository repository = new FileBookRepository();  
private static final FinePolicy finePolicy = new StandardFinePolicy();
```

Location 2: Inside case 2 of manage() (line 44)

// mainClass.management(regNumber);

```
// =====  
// SMELL: DEAD CODE – this call is commented out and never  
// runs. It does nothing here inside case 2.  
// =====  
// mainClass.management(regNumber);  
break;
```

Both lines were commented out and never executed. They reference a Main class that no longer exists in the project.

Refactoring:

Remedy Used: Remove Dead Code

Both commented-out lines were deleted entirely. Commented-out code confuses readers who wonder if it is important or needs to be uncommented. Git version history preserves old code — comments are not a backup system.

LibraryManagement.java & FileBookRepository.java:

3. Duplicate Code

The same hardcoded value was written independently in two separate files:

In LibraryManagement.java → returnBook():

LocalDate.now().minusDays(40)

```
// =====
// SMELL: DUPLICATE CODE – the number 40 is hardcoded here

LocalDate borrowedDate = LocalDate.now().minusDays(daysToSubtract: 40); // keep old log
LocalDate returnDate = LocalDate.now();
int fine = finePolicy.calculateFine(borrowedDate, returnDate);
if (fine > 0)
    System.out.println("Fine: " + fine + " rupees.");
else
    System.out.println(x: "No fine.");
```

In FileBookRepository.java → getBorrowedBooks():

LocalDate.now().minusDays(40)

```
// =====
// SMELL: DUPLICATE CODE – the number 40 is hardcoded here

    borrowed.add(new Book(line, LocalDate.now().minusDays(daysToSubtract: 40)));
}
sc.close();
} catch (Exception e) {
    System.out.println(x: "Error reading borrowed books.");
}
return borrowed;
}
```

The raw number 40 had no explanation in either file. If the value ever needed to change, both files had to be updated. Missing one would cause the two files to behave differently.

Refactoring:

Remedy Used: Replace Magic Number with Named Constant

A named constant `SIMULATED_DAYS_BORROWED = 40` was declared at the top of each file. The raw number 40 was replaced with this constant in both locations. The name explains what the value means, and it is easy to find and update.

In LibraryManagement.java :

```
// SMELL #5 FIXED: Using the named constant instead of raw 40
LocalDate borrowedDate = LocalDate.now().minusDays(SIMULATED_DAYS_BORROWED);
LocalDate returnDate = LocalDate.now();
int fine = finePolicy.calculateFine(borrowedDate, returnDate);
if (fine > 0)
    System.out.println("Fine: " + fine + " rupees.");
```

In FileBookRepository.java :

```

    // SMELL #5 FIXED: Using the named constant instead of raw 40
    borrowed.add(new Book(line, LocalDate.now().minusDays(SIMULATED_DAYS_BORROWED)));
}
sc.close();
} catch (Exception e) {
    System.out.println(x: "Error reading borrowed books.");
}
return borrowed;
}

```

Design patterns

Overview

This report covers the design patterns that were found and applied in the Library Management System project. The previous phase focused on SOLID principles and fixing code smells. In this phase, the code was reviewed to see which design patterns from the course were already present or could be added in a way that made sense.

After going through the code, three patterns were found to be relevant:

- Strategy Pattern (Behavioral) this was already in the code from the SOLID refactoring phase
- Singleton Pattern (Creational) applied to FileBookRepository in this phase
- Factory Method Pattern (Creational) applied by adding a new BookFactory class

Pattern	Category	Status	Where Applied	Files Changed
Strategy	Behavioral	Already existed	FinePolicy + StandardFinePolicy	None (unchanged)
Singleton	Creational	Newly applied	FileBookRepository	FileBookRepository.java, LibraryManagement.java
Factory Method	Creational	Newly applied	BookFactory (new class)	BookFactory.java (new), FileBookRepository.java

Pattern 1: Strategy Pattern

Status: Already existed in the code

Category: Behavioral | Files: FinePolicy.java, StandardFinePolicy.java, LibraryManagement.java

What is the Strategy Pattern?

Strategy Pattern is when you have a task that can be done in different ways, and each way is put in its own separate class. The main class does not directly use any of those classes. It uses an interface instead, so the actual method being used can be changed without touching the main class.

Why was it already present?

When the SOLID principles were applied in the previous phase, the fine calculation was moved out of LibraryManagement and placed behind a FinePolicy interface. At the time the focus was on following the Open/Closed Principle, but looking at the structure now, it already matches the Strategy Pattern:

- FinePolicy is the interface that defines what the task is (calculateFine)
- StandardFinePolicy is the class that actually does the calculation
- LibraryManagement holds a FinePolicy reference — it does not directly use StandardFinePolicy

Before — fine calculation was written directly in LibraryManagement:

```
// The calculation was hardcoded inside LibraryManagement
long daysLate = ChronoUnit.DAYS.between(borrowedDate, returnDate) - 30;
int fine = daysLate > 0 ? (int)(daysLate * 50) : 0;

// To change the fine rule, you had to edit LibraryManagement directly.
// There was no way to swap in a different rule without modifying this class.
```

After — fine calculation moved behind an interface:

```
// FinePolicy.java — defines what the task is
public interface FinePolicy {
    int calculateFine(LocalDate borrowedDate, LocalDate returnDate);
}

// StandardFinePolicy.java — one implementation of the task
public class StandardFinePolicy implements FinePolicy {
    private final int maxDaysWithoutFine = 30;
    private final int finePerDay = 50;
    @Override
    public int calculateFine(LocalDate borrowedDate, LocalDate returnDate) {
        long daysLate = ChronoUnit.DAYS.between(borrowedDate, returnDate) - maxDaysWithoutFine;
        return daysLate > 0 ? (int)(daysLate * finePerDay) : 0;
    }
}

// LibraryManagement.java — talks to the interface, not the class
private static final FinePolicy finePolicy = new StandardFinePolicy();
```

Pattern 2: Singleton Pattern

Status: Newly applied in this phase

Category: Creational | Files changed: FileBookRepository.java, LibraryManagement.java

What is the Singleton Pattern?

Singleton Pattern means only one instance of a class is allowed to exist in the whole program. Instead of creating the object with 'new', there is a static method that always returns the same existing object.

Why was it applied to FileBookRepository?

FileBookRepository reads and writes files on disk. In the original code, it had a public constructor, so any part of the program could do 'new FileBookRepository()' and create a separate instance. If two instances were active at the same time and both tried to write to the same file, the data could get corrupted or one write could overwrite the other.

Making it a Singleton means there will always be only one instance handling the files, which avoids that problem.

Before-----public constructor, no restriction on how many instances can be created:

```
// FileBookRepository.java
public class FileBookRepository implements BookRepository {
    // public constructor by default
    // any class can do: new FileBookRepository()
}

// LibraryManagement.java
private static final BookRepository repository = new FileBookRepository();
// Another class could also do: new FileBookRepository()
// Both would read/write the same files separately
```

After -----Singleton applied

```
public class FileBookRepository implements BookRepository {

    // Step 2: The single instance, stored privately
    private static FileBookRepository instance;

    // Step 1: Private constructor – prevents anyone from doing "new FileBookRepository()"
    private FileBookRepository() {}

    // Step 3: The only way to get this object – always returns the same one
    public static FileBookRepository getInstance() {
        if (instance == null) {
            instance = new FileBookRepository();
        }
        return instance;
    }
}

// LibraryManagement.java
private static final BookRepository repository = FileBookRepository.getInstance();
```

Pattern 3: Factory Method Pattern

Status: Newly applied in this phase

Category: Creational | Files changed: BookFactory.java (new), FileBookRepository.java

What is the Factory Method Pattern?

Factory Method Pattern means having a separate class whose job is to create objects. Instead of writing 'new Book(...)' directly wherever a Book is needed, the code asks a factory class to create it instead.

Why was it applied here?

In the original code, FileBookRepository was calling 'new Book(...)' in two different places inside its methods. This means the repository class was doing two things: reading and saving books from files, and also figuring out how to build Book objects. These are two separate responsibilities.

Also, if the Book class constructor changes later, every place that calls 'new Book(...)' directly would have to be updated. With a factory, there is only one place that creates Book objects, so only that one place would need to change.

Before — Book objects created directly inside FileBookRepository:

```
public List<Book> getAvailableBooks() {  
    List<Book> books = new ArrayList<>();  
    try {  
        File file = new File(availableFile);  
        if (!file.exists())  
            file.createNewFile();  
        Scanner sc = new Scanner(file);  
        while (sc.hasNextLine()) {  
            String line = sc.nextLine();  
            if (!line.isEmpty())  
                books.add(new Book(line));  
        }  
        sc.close();  
    } catch (Exception e) {  
        System.out.println(x: "Error reading available books.");  
    }  
    return books;  
}
```

After — new BookFactory.java class handles Book creation:

```
libraryV2 > J Bookfactory.java > BookFactory
2/ // =====
28 public class BookFactory {
29
30     // Creates a Book that is sitting on the shelf (no borrow date needed)
31     public static Book createAvailableBook(String title) {
32         return new Book(title);
33     }
34
35     // Creates a Book that has already been borrowed (needs a borrow date)
36     public static Book createBorrowedBook(String title, LocalDate borrowedDate) {
37         return new Book(title, borrowedDate);
38     }
39 }
```

Architectural Pattern: Pipe and Filter

What is Pipe and Filter?

Pipe and Filter is an architectural pattern that structures a process as a sequence of independent stages:

- Source: where raw data enters the pipeline
- Filters: independent stages that validate or transform data one step at a time
- Pipes: the connections that pass data from one filter to the next
- Sink: the final destination where approved data is stored

Where It Is Applied in UMS

The Pipe and Filter pattern is applied in the student registration subsystem (Main.java). Raw user input passes through 10 sequential validation filters before being written to file storage.

The 10 Validation Filters

Filter	Validation Rule
Filter 1 — Name	Min. 3 characters, letters only
Filter 2 — Password	Min. 8 characters
Filter 3 — Degree	Must equal "BSCS"
Filter 4 — Gender	MALE or FEMALE only
Filter 5 — Date of Birth	Regex DD-MM-YYYY

Filter 6 — Student CNIC	Regex #####-#####-#
Filter 7 — Email	Must match @gmail.com pattern
Filter 8 — Contact Number	Regex 03xx-xxxxxxx, exactly 12 chars
Filter 9 — Father's Name	Min. 3 characters, letters only
Filter 10 — Father's CNIC	Regex #####-#####-#
Sink	Writes to ids.txt + students_data/<regNumber>.txt

Diagram

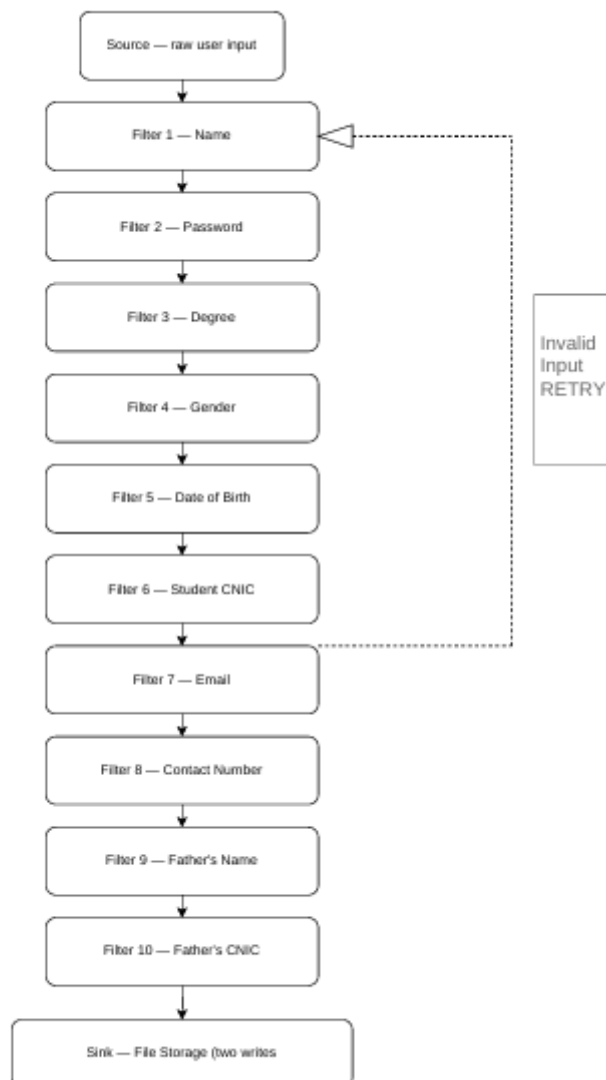


Figure: Pipe and Filter — UMS Student Registration Pipeline

